Kubb Coach: Statistics, Milestones, and Player Level Systems Analysis

Date: March 1, 2026

Purpose: Comprehensive documentation of statistics tracking, personal records, milestones, and player progression systems

Table of Contents

1. Statistics Overview
2. Personal Records (Personal Bests)
3. Milestones
4. Player Level System
5. Streaks
6. View Hierarchy & User Journey
7. Session Data Inclusion
8. Calculation Details

Statistics Overview

Main Statistics Views

1. StatisticsView (`Views/Statistics/StatisticsView.swift`)

The primary statistics interface with three sections:

- **Dashboard** (Default view)
  - Overall session count
  - Overall accuracy
  - Total throws
  - Current streak
  - Accuracy trend chart (last 15 sessions)

    - Insights section
- **Trophy Room**
    - Personal Bests Section
    - Milestones Section
- **Deep Dive**
    - Time range filter: Week / Month / All Time
    - Phase filter: All / 8 Meters / 4M Blasting / Inkasting
    - Phase-specific statistics sections
    - Personal Bests (filtered by phase)
    - Milestones

Phase-Specific Statistics Sections

2. TrainingOverviewSection (`Views/Statistics/TrainingOverviewSection.swift`)

Shown when "All" phases selected in Deep Dive:

- **Streak Overview**
    - Current streak (with live status)
    - Longest streak ever
    - Motivational messages
- **8 Meter Training Overview**
    - Total sessions
    - Recent accuracy (last 4 sessions)
    - Overall accuracy
    - Accuracy trend (recent vs overall delta)
- **4 Meter Blasting Overview**
    - Total sessions
    - Recent score (last 4 sessions)
    - Overall score
    - Score trend (recent vs overall delta)

3. BlastingStatisticsSection (`Views/Statistics/BlastingStatisticsSection.swift`)

4 Meter Blasting-specific view:

- **Key Metrics**
    - Total sessions

    - Average session score (all 9 rounds)
    - Best session score
    - Under par rounds count
- **Score Trend Chart**
    - Line chart of session scores over time
    - Par line at 0
    - Trend indicator (Improving/Declining/Stable)
- **Per-Round Performance Chart**
    - Bar chart showing average score by round number (R1-R9)
    - Shows which rounds are typically easier/harder
- **Personal Records**
    - Best session score
    - Best single round (score + which round)
    - Most kubbs knocked down in a session
    - Under par rounds count

4. InkastingStatisticsSection (`Views/Statistics/InkastingStatisticsSection.swift`)

Inkasting Drilling-specific view with advanced metrics:

- **Mode Selector**: All / 5-Kubb / 10-Kubb
- **Key Metrics**
    - Total sessions
    - Consistency score (% of rounds with 0 outliers)
    - Average core area (cluster without outliers)
    - Best core area
    - Average total spread (including outliers)
    - Average outliers per round
- **Cluster Area Trend Chart**
    - Line chart of average cluster area over time
    - Lower is better (tighter grouping)
- **Total Spread Trend Chart**
    - Line chart of total spread radius over time
    - Shows overall consistency including outliers
- **Outlier Trend Chart**
    - Line chart of average outliers per round
    - Target line at 0 (perfect)
- **Consistency Analysis**

- Perfect rounds count (0 outliers)
- Spread ratio (total spread / core cluster)

5. AccuracyTrendChart (`Views/Statistics/AccuracyTrendChart.swift`)

Reusable chart component:

- Shows last 15 sessions
- Line chart with points
- Y-axis: 0-100% accuracy
- X-axis: Date (month/day)
- Can be filtered by phase

Personal Records (Personal Bests)

Data Model

File: `Models/PersonalBest.swift`

```
@Model
final class PersonalBest {
    var id: UUID
    var category: BestCategory
    var phase: TrainingPhase?  // nil = all phases
    var value: Double
    var achievedAt: Date
    var sessionId: UUID
}
```

Categories

Category	Display Name	Icon	Unit	Phase
`highestAccuracy`	Highest Accuracy	target	%	Per-phase
`lowestBlastingScore`	Best Blasting Score	trophy.fill	(score)	Blasting only
`longestStreak`	Longest Streak	flame.fill	days	All phases

Category	Display Name	Icon	Unit	Phase
`mostConsecutiveHits`	Hit Streak	arrow.up.right	hits	All phases
`perfectRound`	Perfect Round	star.circle.fill	✓	All phases
`perfectSession`	Perfect Session	crown.fill	(none)	All phases
`mostSessionsInWeek`	Most Sessions (Week)	calendar	sessions	All phases
`tightestInkastingCluster`	Tightest Cluster	scope	cm^2	Inkasting only

Checking Logic

File: `Services/PersonalBestService.swift`

Personal bests are checked **after every session completion** via `TrainingSessionManager.completeSession()`.

Accuracy Best (`highestAccuracy`)

- **Scope:** Per phase
- **Calculation:** Session accuracy (hits/throws × 100)
- **Logic:** Creates/updates if current session accuracy > existing best for that phase

Blasting Score Best (`lowestBlastingScore`)

- **Scope:** 4M Blasting only
- **Calculation:** Total session score (sum of all 9 rounds)
- **Logic:** Creates/updates if current session score < existing best (lower is better)

Perfect Round (`perfectRound`)

- **Scope:** All phases, one-time achievement
- **Calculation:** Any round with 100% accuracy
- **Logic:** Created once when first achieved

Perfect Session (`perfectSession`)

- **Scope:** All phases, one-time achievement per phase
- **Calculation:** Session with 100% accuracy

- **Logic:** Created once per phase when first achieved

Consecutive Hits (`mostConsecutiveHits`)

- **Scope:** All phases, cross-session
- **Calculation:** Longest streak of consecutive hits across all throws in session
- **Logic:** Creates/updates if current streak ≥ 5 and > existing best

Inkasting Cluster (`tightestInkastingCluster`)

- **Scope:** Inkasting only
- **Calculation:** Smallest cluster area (square meters) from any round with 0 outliers
- **Logic:** Creates/updates if current best area < existing best

Display Location

View: `Views/Statistics/PersonalBestsSection.swift`

- Displayed in **Trophy Room** section of StatisticsView
- Also appears in **Deep Dive** section (filtered by selected phase)
- Grid layout with cards for each category
- Shows "—" for unachieved records
- Gold highlight for achieved records

Milestones

Data Model2

Files:

- `Models/Milestone.swift` (definitions)
- `Models/EarnedMilestone.swift` (persistence)

```swift
struct MilestoneDefinition: Identifiable {
    let id: String
    let title: String
    let description: String
    let icon: String
    let category: MilestoneCategory
    let threshold: Int
    let color: Color
}

@Model
final class EarnedMilestone {
    var id: UUID
    var milestoneId: String
    var earnedAt: Date
    var sessionId: UUID?
    var hasBeenSeen: Bool  // For overlay display
}
```

Categories2

1. Session Count (`sessionCount`)

Progressive milestones based on total completed sessions:

- **First Steps** (1 session) - Blue
- **Getting Started** (5 sessions) - Blue
- **Dedicated** (10 sessions) - Gold
- **Committed** (25 sessions) - Gold
- **Veteran** (50 sessions) - Orange
- **Century** (100 sessions) - Gold

2. Streak (`streak`)

Based on consecutive training days:

- **Hat Trick** (3 days) - Orange
- **Full Week** (7 days) - Gold
- **Fortnight** (14 days) - Gold
- **Monthly Master** (30 days) - Purple

3. Performance (`performance`)

One-time achievements:

- **Sharpshooter** (80% accuracy in session) - Green
- **Perfect Round** (100% accuracy in round) - Gold
- **Perfect Session** (100% accuracy in session) - Gold
- **King Slayer** (throw at king successfully) - Purple
- **Under Par** (blasting round with negative score) - Green
- **Eagle Eye** (5 consecutive hits) - Green
- **Untouchable** (10 consecutive hits) - Gold

Checking Logic2

File: `Services/MilestoneService.swift`

Milestones are checked **after every session completion** via `TrainingSessionManager.completeSession()` .

Session Count Milestones

- **Trigger:** Checks if total session count equals any threshold
- **Data:** Uses combined local + cloud sessions
- **Logic:** Creates milestone if threshold hit and not previously earned

Streak Milestones

- **Trigger:** Checks if current streak equals any threshold
- **Calculation:** Uses `StreakCalculator.currentStreak()`
- **Logic:** Creates milestone if threshold hit and not previously earned

Performance Milestones

Milestone	Condition	Data Source
Sharpshooter	session.accuracy ≥ 80%	Session aggregate
Perfect Round	Any round with accuracy == 100%	Round data
Perfect Session	session.accuracy == 100%	Session aggregate
King Slayer	kingThrowCount > 0 && kingThrowAccuracy > 0	Session aggregate

Milestone	Condition	Data Source
Under Par	phase == blasting && totalSessionScore < 0	Session aggregate
Eagle Eye	Max consecutive hits ≥ 5	Throw records
Untouchable	Max consecutive hits ≥ 10	Throw records

Display Locations

MilestonesSection (`Views/Statistics/MilestonesSection.swift`)

- Shown in **Trophy Room** and **Deep Dive** sections
- Grouped by category (Session Progress, Training Streaks, Performance)
- Horizontal scrollable cards
- Locked (gray) for unearned, colored for earned
- Checkmark for earned, lock icon for locked

MilestoneAchievementOverlay (`Views/Components/MilestoneAchievementOverlay.swift`)

- Modal overlay shown **immediately after session completion**
- Displays newly earned milestones
- Animated icon rotation
- Sound + haptic feedback
- Marks milestone as "seen" when dismissed

Player Level System

Data Structure

File: `Services/PlayerLevelService.swift`

```swift
struct PlayerLevel {
    let levelNumber: Int
    let name: String
    let subtitle: String
    let currentXP: Int
    let xpForCurrentLevel: Int
    let xpForNextLevel: Int
    let totalSessions: Int
    var xpProgress: Double  // 0.0 to 1.0
    var isMaxLevel: Bool    // Level 60+
}
```

Level Tiers (Ranks)

Level Range	Name (Swedish)	Subtitle (English)	Icon
1-5	Nybörjare	Beginner	figure.walk
6-15	Spelare	Player	figure.run
16-30	Kastare	Thrower	bolt.fill
31-50	Viking	Viking	shield.fill
51-60	Kung	King	crown.fill

XP Calculation

XP is awarded **per completed session** based on training mode.

8 Meters (Standard) Mode

```
Total XP = (totalThrows × 0.2) + (totalHits × 0.2)
```

- 0.2 XP per throw
- 0.2 XP per hit
- Example: 60 throws, 45 hits = 12 + 9 = **21 XP**

4 Meters Blasting Mode

```
Total XP = (roundCount × 0.3) + (underParRounds × 0.3)
```

- 0.3 XP per round (9 rounds total)
- 0.3 XP bonus for each under-par round (score < 0)
- Example: 9 rounds, 3 under par = 2.7 + 0.9 = **3.6 XP**

Inkasting Drilling Mode

```
Total XP = Σ(kubbCount × 0.05 × multiplier)
where multiplier = 2.0 if outlierCount == 0, else 1.0
```

- 0.05 XP per kubb thrown
- **Double XP** for perfect rounds (zero outliers)
- Example: 3 rounds of 10 kubbs, 1 perfect = (10×0.05) + (10×0.05×2) + (10×0.05) = 0.5 + 1.0 + 0.5 = **2.0 XP**

Level Progression

XP thresholds increase with level:

Level	XP Required (Total)	XP for This Level
1	0	0
2-5	50, 100, 150, 200	50 each
6-15	300, 400, ..., 1200	100 each
16-30	1400, 1600, ..., 4200	200 each
31-50	4550, 4900, ..., 11200	350 each
51-60	11700, 12200, ..., 16200	500 each

Display Locations2

PlayerCardView (`Views/Components/PlayerCardView.swift`)

- **Location:** Top of HomeView ("The Lodge")
- **Displays:**
  - Level icon with gradient background (tier-based color)
  - Rank name (Swedish)
  - Level number
  - Total session count

    - Current streak (if > 0)
    - XP progress bar (to next level)
    - Current XP / Next level XP
    - Progress percentage

LevelUpCelebrationOverlay (`Views/Components/LevelUpCelebrationOverlay.swift`)

- **Trigger:** After session completion when level increases
- **Two types:**
    i. **Regular Level Up:** Same rank, new level number
    ii. **Rank Up:** New rank tier (special animation with crown, gold colors)
- **Features:**
    - Full-screen modal
    - Animated icon
    - Old → New level display
    - Sound effect (different for regular vs rank up)
    - Haptic feedback

Calculation Context

Computation: Always uses **all completed sessions** (local + cloud) via:

```
PlayerLevelService.computeLevel(from: allSessions)
```

- Includes all training phases
- Cloud sessions synced from Apple Watch
- Recalculated on every view render

Streaks

Data Structure3

File: `Utilities/StreakCalculator.swift`

Streaks are calculated **dynamically** from session dates (not stored).

Current Streak

Function: `StreakCalculator.currentStreak(from: [SessionDisplayItem]) -> Int`

Logic:

1. Extract unique calendar days from all session dates
2. Check if today OR yesterday has a session
   - If neither: streak is **0** (broken)
   - If yes: start counting backwards
3. Count consecutive days backwards from most recent session
4. Return total consecutive days

Example:

- Sessions on: Mar 1, Feb 29, Feb 28, Feb 26
- Today: Mar 1
- Result: **3 days** (Mar 1, Feb 29, Feb 28)

Longest Streak

Function: `StreakCalculator.longestStreak(from: [SessionDisplayItem]) -> Int`

Logic:

1. Extract unique calendar days from all sessions
2. Sort chronologically
3. Iterate through dates, tracking consecutive sequences
4. Return maximum consecutive sequence found

Example:

- Sessions on: Jan 1, Jan 2, Jan 3, Jan 10, Jan 11, Jan 15
- Sequences: [3 days], [2 days], [1 day]
- Result: **3 days**

Display Locations3

1. **Dashboard Section** (StatisticsView)
   - Shows current streak only
   - Part of the 4-metric summary grid

2. **Training Overview Section**
    - Current streak with visual indicator
    - Longest streak comparison
    - Motivational messages based on status
3. **PlayerCardView** (HomeView)
    - Current streak with flame icon
    - Scaled flame based on streak length (1-3 days: 1.0x, 4-7: 1.15x, 8-14: 1.3x, 15+: 1.5x)

View Hierarchy & User Journey

Navigation Structure

```
MainTabView
├── Home Tab (HomeView)
│   ├── PlayerCardView
│   │   └── Shows: Level, XP, Streak
│   ├── Training Mode Cards
│   └── Recent Performance Sparkline
│
├── Statistics Tab (StatisticsView)
│   ├── Dashboard Section (Default)
│   │   ├── Your Numbers (4 metrics)
│   │   ├── Accuracy Trend Chart
│   │   └── Insights
│   │
│   ├── Trophy Room Section
│   │   ├── PersonalBestsSection
│   │   └── MilestonesSection
│   │
│   └── Deep Dive Section
│       ├── Time Range Picker (Week/Month/All Time)
│       ├── Phase Picker (All/8M/4M/Inkasting)
│       ├── Phase-specific statistics
│       │   ├── TrainingOverviewSection (All)
│       │   ├── BlastingStatisticsSection (4M)
│       │   ├── InkastingStatisticsSection (Inkasting)
│       │   └── 8M metrics + AccuracyTrendChart (8M)
│       ├── PersonalBestsSection (filtered)
│       └── MilestonesSection
│
└── History Tab
    └── SessionHistoryView (detailed session records)
```

Post-Session Flow

```
Session Complete
└──> TrainingSessionManager.completeSession()
     ├── Save completedAt timestamp
     ├── PersonalBestService.checkForPersonalBests()
     │   └── Creates PersonalBest records if thresholds met
     ├── MilestoneService.checkForMilestones()
     │   └── Creates EarnedMilestone records if thresholds met
     └── Save session with newPersonalBests & newMilestones IDs

Session Completion View
├── Shows session summary
├── XP gained (if computed)
└── Overlays (shown sequentially):
    ├── LevelUpCelebrationOverlay (if level increased)
    └── MilestoneAchievementOverlay (for each new milestone)
```

Session Data Inclusion

What Sessions Are Included?

All statistics, levels, and milestones use **all completed sessions** from:

1. **Local Sessions (iOS device)**
   - Stored in SwiftData
   - Queried with: `completedAt != nil`
2. **Cloud Sessions (Apple Watch)**
   - Synced via CloudKit
   - Fetched from `CloudKitSyncService`
   - Cached locally in `CachedCloudSession`

Combined Session Data

Sessions are unified via `SessionDisplayItem` enum:

```
enum SessionDisplayItem {
    case local(TrainingSession)
    case cloud(CloudSession)
}
```

This allows:

- Unified sorting by date
- Phase filtering across devices
- Accurate XP/level calculation
- Correct streak counting

Refresh Behavior

- **Statistics Tab:** Pull-to-refresh fetches cloud sessions
- **Home View:** Loads cloud sessions on appear
- **Cloud sessions:** Cached for performance, force-refreshable

Calculation Details

Accuracy Calculations

Session Accuracy

```
accuracy = (totalHits / totalThrows) × 100
```

- Includes all throw types (kubbs + king)
- Calculated per session
- Displayed with 1 decimal place

Round Accuracy

```
accuracy = (round.hits / round.throwRecords.count) × 100
```

- Per round calculation
- Used for "Perfect Round" milestone (100%)

King Throw Accuracy

```
kingThrowAccuracy = (kingHits / kingThrowCount) × 100
```

- Separate calculation for king-only throws
- Used for "King Slayer" milestone

Blasting Score Calculations

Round Score

```
score = throwCount - par
where par = 6 (for 5 kubbs + king)
```

- Lower is better
- Negative score = under par (good!)
- Examples:
  - 5 throws: score = -1 (eagle)
  - 6 throws: score = 0 (par)
  - 8 throws: score = +2 (over par)

Session Score

```
totalSessionScore = sum(all 9 round scores)
```

- Example: [-1, 0, +1, -1, 0, 0, +2, -1, 0] = **0** (at par)

Inkasting Metrics

Note: iOS only (requires Vision framework)

Cluster Area

- Calculated by `InkastingAnalysisService` using Vision framework
- Measures the convex hull area of kubbs **excluding outliers**
- Stored in square meters, displayed in cm^2
- Lower = tighter grouping = better

Total Spread Radius

- Maximum distance from centroid to any kubb (including outliers)
- Stored in meters
- Shows overall consistency

Outlier Count

- Kubbs detected as statistical outliers from the core cluster
- 0 outliers = perfect round = double XP

Consistency Score

```
consistencyScore = (roundsWithZeroOutliers / totalRounds) × 100
```

- Primary metric for inkasting improvement
- Goal: increase percentage over time

Spread Ratio

```
spreadRatio = totalSpreadRadius / clusterCoreRadius
```

- How much larger total spread is compared to core cluster
- Values near 1.0 = tight, minimal outliers
- Higher values = more scattered throws

Trend Calculations

Used in Deep Dive charts to show improvement/decline:

```
recentAverage = average(last N/2 sessions or max 3-5)
olderAverage = average(first N/2 sessions or max 3-5)
delta = recentAverage - olderAverage

if delta > threshold: "Improving" or "Declining"
else: "Stable"
```

Thresholds:

- Accuracy: ±1%

- Blasting score: ±2 points
- Cluster area: ±0.5 m^2
- Outliers: ±0.3

Summary of Key Findings

Statistics

- ✅ Three-tiered interface (Dashboard, Trophy Room, Deep Dive)
- ✅ Phase-specific metrics for each training mode
- ✅ Time-range filtering (Week/Month/All Time)
- ✅ Trend analysis with improvement indicators
- ✅ Combines local + cloud session data

Personal Bests

- ✅ 8 categories across all training modes
- ✅ Phase-specific and cross-phase records
- ✅ Automatically checked after every session
- ✅ Persistent storage in SwiftData
- ✅ Displayed in Trophy Room and Deep Dive (filtered)

Milestones2

- ✅ 3 categories: Session Count, Streak, Performance
- ✅ 18 total milestone definitions
- ✅ One-time achievements (not progressive)
- ✅ Post-session modal overlay for new achievements
- ✅ Visual progression tracking (locked vs earned)

Player Level

- ✅ 60 levels across 5 rank tiers
- ✅ XP formulas customized per training mode
- ✅ Bonus XP for performance (under par, zero outliers)
- ✅ Special celebration for rank-ups
- ✅ Visual XP progress bar

- ✅ Prominently displayed on home screen

Streaks2

- ✅ Current streak (active if trained today/yesterday)
- ✅ Longest streak (all-time record)
- ✅ Used for milestone checking
- ✅ Multiple display locations with visual emphasis
- ✅ Dynamically calculated from session dates

Technical Architecture

Services

- **PlayerLevelService** - XP calculation and level determination
- **PersonalBestService** - PB checking and persistence
- **MilestoneService** - Milestone checking and tracking
- **TrainingSessionManager** - Orchestrates post-session checks
- **CloudKitSyncService** - Cloud session synchronization
- **StreakCalculator** - Streak calculations

Models

- **TrainingSession** - Core session data
- **TrainingRound** - Round-level data
- **ThrowRecord** - Individual throw data
- **PersonalBest** - PB records
- **EarnedMilestone** - Milestone achievements
- **InkastingAnalysis** - Inkasting metrics (iOS only)

Views

- **StatisticsView** - Main stats interface
- **PersonalBestsSection** - PB grid display
- **MilestonesSection** - Milestone cards
- **PlayerCardView** - Level/XP display

- **Various Statistics Sections** - Phase-specific views
- **Celebration Overlays** - Achievement notifications

Recommendations for Future Enhancement

Based on this analysis, potential areas for improvement:

1. **Personal Bests:**
    - Consider adding "weekly best" time-limited records
    - Add "longest session" PB category
    - Include PB history/progression tracking
2. **Milestones:**
    - Consider progressive milestones (multiple tiers per achievement)
    - Add social/sharing capabilities for milestone unlocks
    - Milestone progress indicators (e.g., "3/5 sessions to Dedicated")
3. **Player Level:**
    - Add level-based unlocks or rewards
    - Show XP breakdown by session in history
    - Level leaderboards (if social features added)
4. **Statistics:**
    - Add weekly/monthly email reports
    - Export statistics to CSV/PDF
    - Custom date range selection (not just Week/Month/All)
    - Comparison view (this week vs last week)
5. **Streaks:**
    - Streak freeze/protection (1 missed day allowed)
    - Streak recovery notifications
    - Historical streak graph

End of Analysis